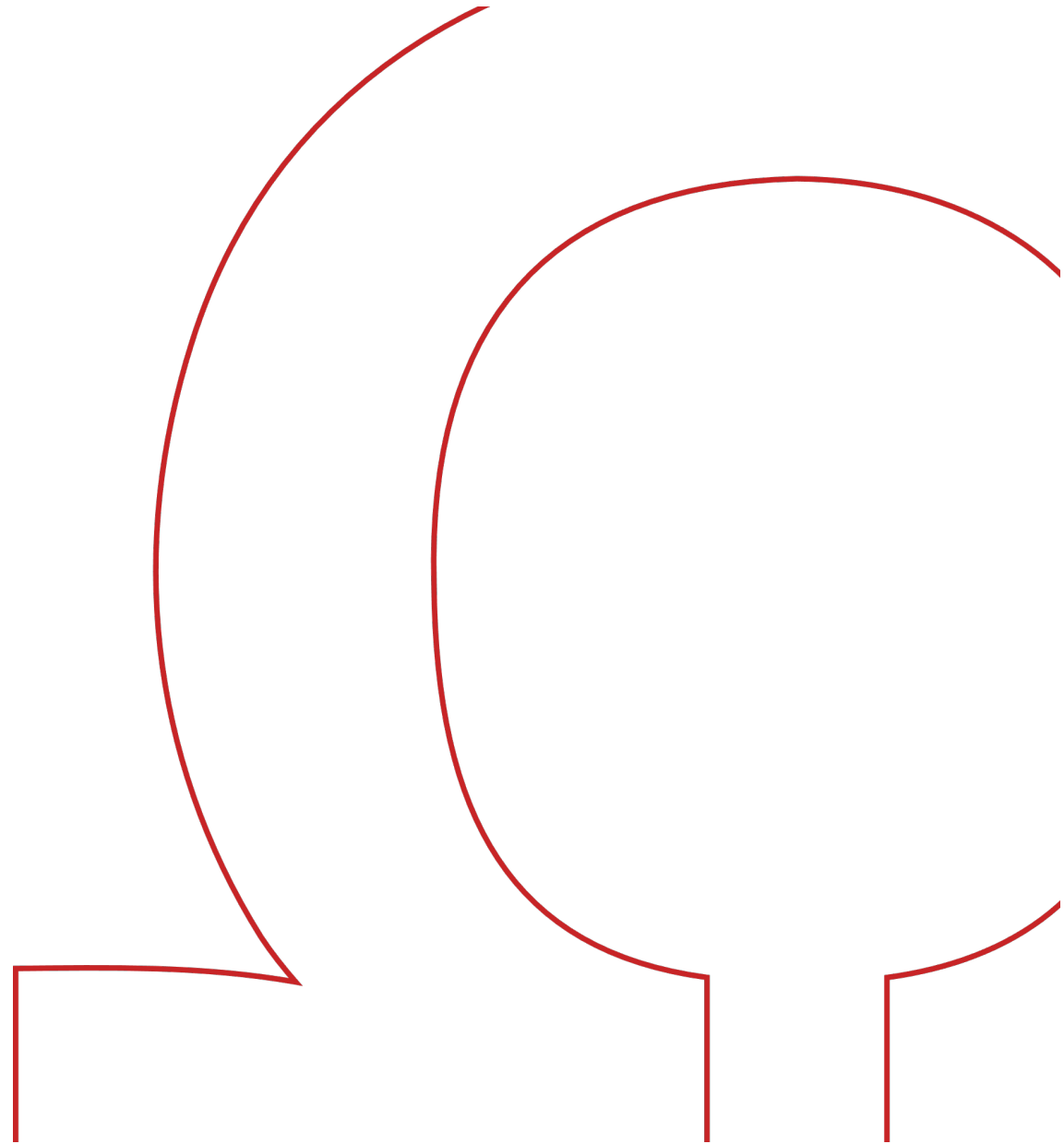


Prog III / A

Software Engineering



Roll Out

Roll Out

Transport vom Entwicklungsteam zum Anwender



Roll Out

Transport vom Entwicklungsteam zum Anwender

Entwicklungssystem

Software



Oft: Entwicklungs-PC
relativ wenige Daten
geringe Verfügbarkeit
Performance für 1 Person
Lokale Zugriffe

Integrationssystem



Oft: Teamserver
geringe Verfügbarkeit
Performance für wenige
Personen
Lokale Zugriffe

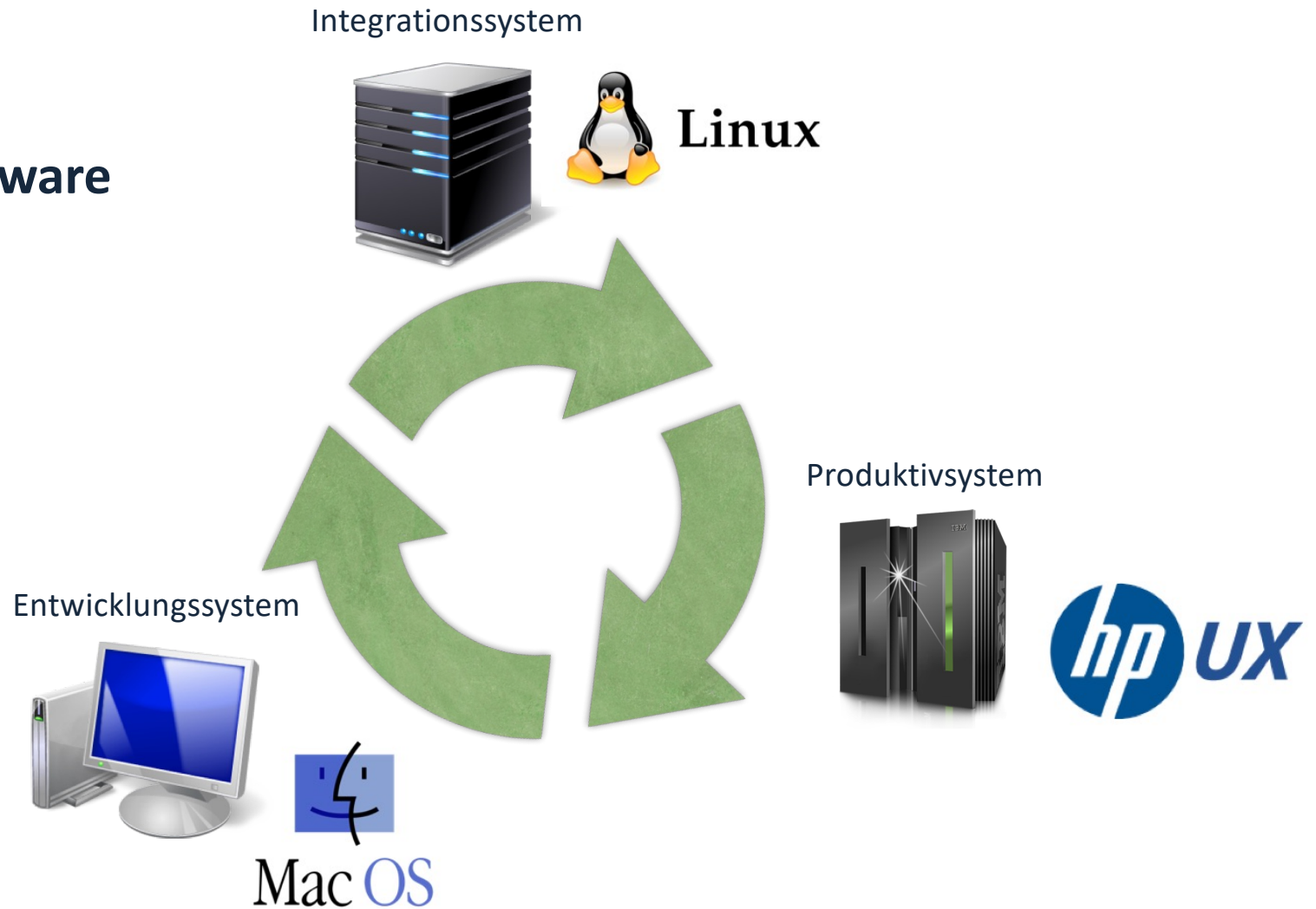
Produktivsystem



Rechenzentrum oder Cloud
hohe Verfügbarkeit
Performance für viele
Personen
Globale Zugriffe

Roll Out

Diversität der Hardware
und Software



Roll Out

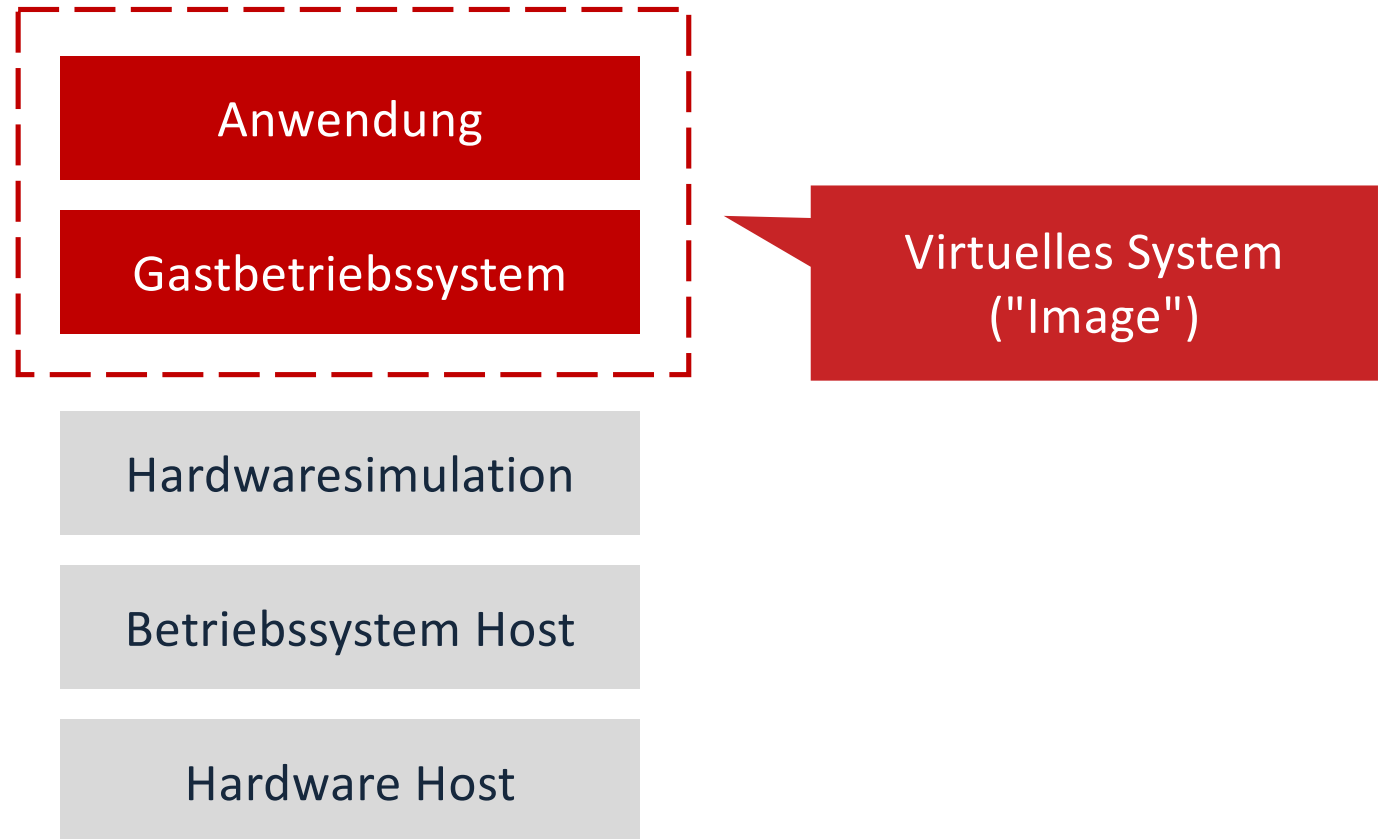
Virtualisierung

Grundidee:

- Definition eines idealtypischen Rechners (Hardware und Software)
- Dieser idealtypische Rechner wird von allen beteiligten Host-Systemen (Entwicklungssystem, Integrationssystem, Produktivsystem) "simuliert"
- Er existiert also nicht tatsächlich, sondern ist nur virtuell vorhanden (Virtualisierung)
- Existierende Hardware kann damit gut und flexibel ausgelastet werden

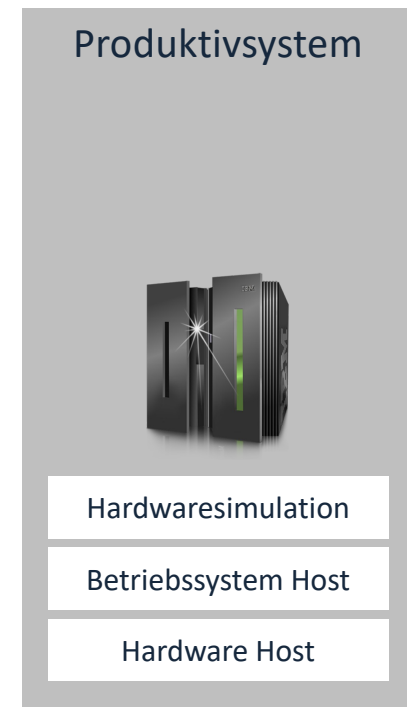
Roll Out

Virtualisierung



Roll Out

Transport in einer virtualisierten Umgebung



Roll Out

Virtualisierung

Probleme:

- Die Images sind recht voluminös, da ein vollständiges Betriebssystem enthalten ist, und sind damit nur aufwändig von Rechner zu Rechner zu kopieren
- Der Start eines virtuellen Rechners dauert lange, da ein "Betriebssystem im Betriebssystem" geladen werden muss
- Die Betriebssysteme in den Images müssen aktiviert werden (Windows!) und auf einem aktuellen Patchlevel gehalten werden

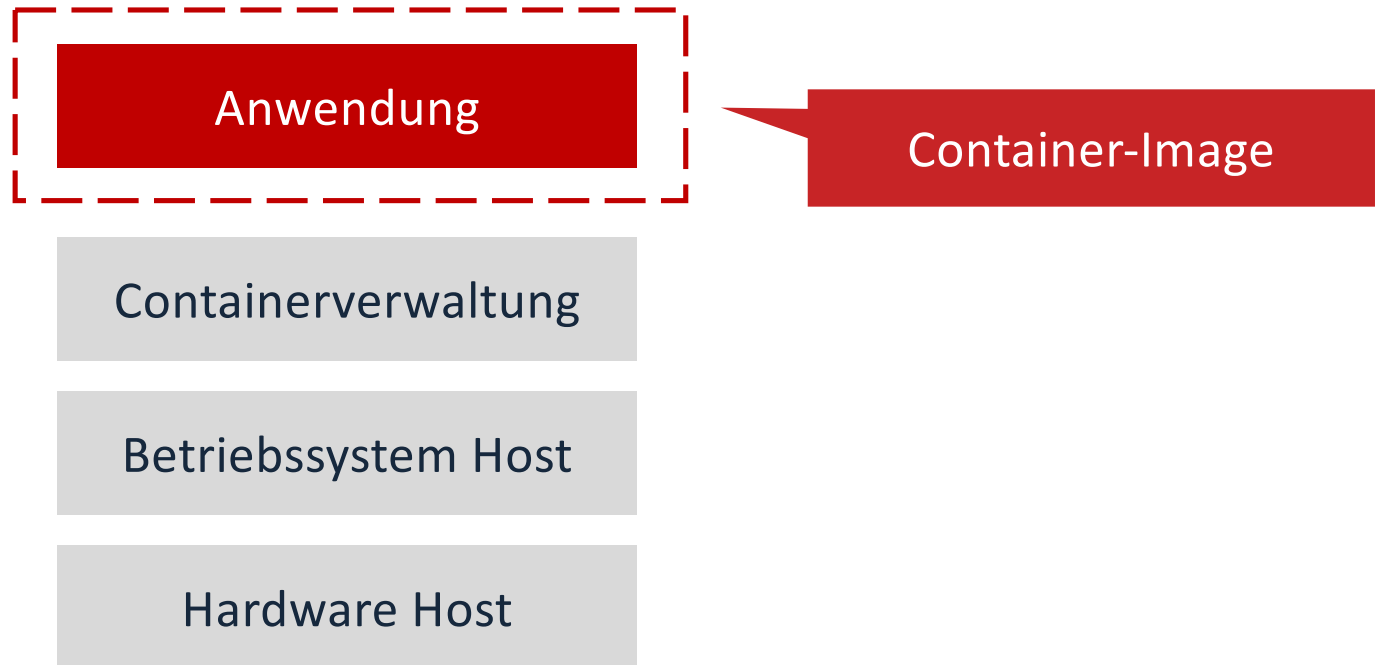
Roll Out

Container

Grundidee:

- Voraussetzung: gemeinsame Betriebssystemfamilie (z.B. Unix)
- Auf ein Gastbetriebssystem kann verzichtet werden, nur abweichende/zusätzliche Bibliotheken kommen in das Image
- Die Container-Images sind damit wesentlich kleiner und lassen sich leicht zwischen Systemen transportieren
- Container lassen sich in wenigen Sekunden starten

Roll Out Container



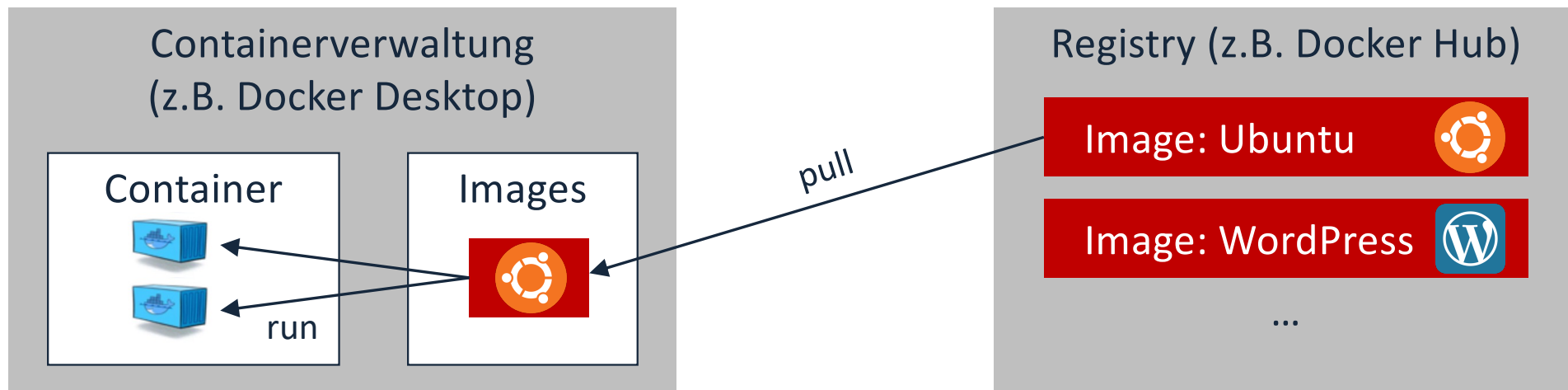
ohm

Roll Out

Docker



Docker ist die bekannteste Containerverwaltung. Die Nutzung ist kostenfrei für kleine Unternehmen, Ausbildung und nicht-kommerzielle Projekte.



Vererbung

Vererbung

Generalisierung vs. Spezialisierung



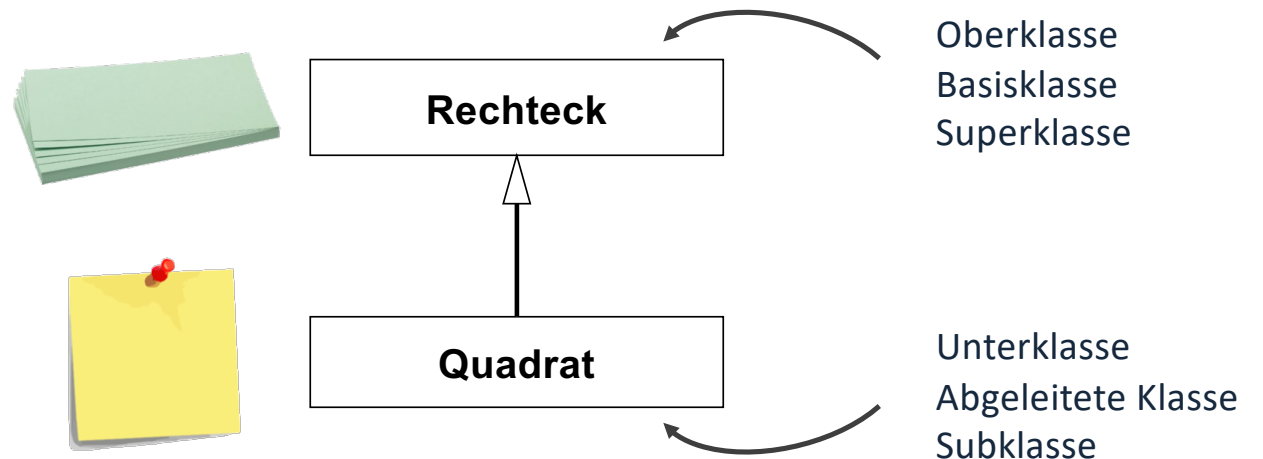
Eine Generalisierung abstrahiert von konkreten Ausprägungen und liefert ein allgemeineres Konzept.

Ein allgemeines Prinzip wird spezialisiert, indem vorhandene Freiheitsgrade durch genaue Vorgaben ersetzt werden.

Vererbung

Generalisierung vs. Spezialisierung

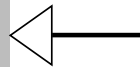
Generalisierung und Spezialisierung können in Java durch eine hierarchische Anordnung der Klassen dargestellt werden.



Vererbung

Vererbung in Java

```
class Rectangle
{
    //--- Attribute
    double a; // Seitenlänge a
    double b; // Seitenlänge b
    //--- Konstruktor
    Rectangle(double a, double b)...
    //--- Methoden
    double getA() { ... }
    double getB() { ... }
}
```



```
class Square extends Rectangle
{
    //--- Attribute
    double a; // Seitenlänge a
    double b; // Seitenlänge b
    //--- Methoden
    double getA() { ... }
    double getB() { ... }
}
```

Konstrukturen werden nicht vererbt!

Vererbung

Ich packe meinen Koffer...

- Jeder neue Koffer muss mindestens enthalten, was der Vorgänger auch enthalten hat.
- Es darf nichts weggelassen werden.
- Alle alten Bestandteile müssen in der gleichen Reihenfolge gepackt werden.



Vererbung

Ich packe meinen Koffer...



Auf meinem Flug ist mein alter Koffer verloren gegangen. Die Fluggesellschaft bietet mir einen neuen Koffer an, der nach den "Ich packe meinen Koffer"-Regeln nach meinem alten Koffer gepackt wurde.

Soll ich das Angebot annehmen? Warum?

Vererbung

Ich packe meinen Koffer...



JA!

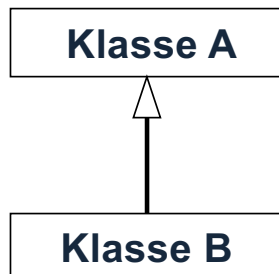
Natürlich nehme ich den neuen Koffer:

- Ich kann damit generell alles machen, was ich schon mit meinem alten Koffer machen konnte!
- Ich kann vielleicht sogar noch mehr Spezielleres damit anstellen!

Der neue Koffer ist eine **Spezialisierung** meines alten Koffers. Er ist als solche in der Lage, jederzeit meinen alten Koffer zu ersetzen.

Vererbung

Liskovsches Substitutionsprinzip



Jede Klasse B, die in einer Vererbungshierarchie unterhalb einer Klasse A angesiedelt ist, ist eine **Spezialisierung** von A.

Instanzen von B sollten jederzeit wie eine Instanz von A auftreten und somit eine Instanz der Oberklasse vollständig ersetzen (**substituieren**) können.



Barbara Liskov

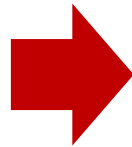
Vererbung

Upcast

```
class A
{
    public void methodeVonA()
    ...
}
```



```
class B extends A
{
    ...
}
```



```
// Vereinbarung einer
// Referenz auf den Typ A

A a = null;

// Zuweisung einer Instanz
// von B ist zulässig, weil
// B vollständig substituiert.

a = new B(); // Upcast

// Der Nutzer der Referenz
// soll anschließend nicht
// merken, dass er mit einer
// Referenz auf B arbeitet.

a.methodeVonA();
```

Die Zuweisung des „falschen“ Typs an eine Referenz der Oberklasse wird vom Compiler akzeptiert. Man spricht von einer impliziten Typumwandlung, einem **Upcast**.

Vererbung

Downcast

Was die Situatiton?

Wir halten eine Referenz auf eine Oberklasse, wissen aber, dass aktuell die Referenz auf eine Instanz einer ganz bestimmte Unterklasse verweist.

Woher wissen wir das?

Die Information kann z.B. dadurch vorhanden sein, weil wir selbst vorher die Referenz entsprechend befüllt haben oder weil wir den Typ der Referenz zur Laufzeit prüfen.

Was wollen wir?

Wir möchten auf die spezielleren Eigenschaften der Instanz zugreifen, die uns mit der allgemeinen Referenz vom Typ der Oberklasse versperrt sind.

Wie gehen wir vor?

Für den Zugriff auf die spezielleren Eigenschaften ist ein konkreterer Referenztyp notwendig. Wir müssen also die allgemeinere Referenz herunterbrechen auf eine konkretere Referenz (**Downcast**).

Vererbung

Downcast



```
// Die folgende Methode soll zum übergebenen  
// Objekt eine möglichst genaue Beschreibung  
// ausgeben.
```

```
public void printInfo(Object o)  
{  
    if (o instanceof Exception)  
    {  
        Exception e = (Exception) o;  
        System.out.println(e.getMessage());  
        return;  
    }  
  
    System.out.println(o.toString());  
}
```

Ist in der Referenz o
eine Exception oder
eine Unterklasse von
Exception enthalten?

Falls ja: Downcast und
Aufruf der spezielleren
Methode

Falls nein: Aufruf der
generelleren Methode

Bei einem **Downcast** kann es zu einer Exception kommen, falls der tatsächlich referenzierte Typ nicht mit dem **Downcast**-Ziel zusammenpasst. Die Überprüfung mit `instanceof` kann einen **Downcast** absichern.

Vererbung

Downcast

Vorsicht „Code Smell“:
Die Verwendung von Downcasts deutet auf ein Designproblem hin!



Warum?

Offensichtlich ist es für den Nutzer der Referenz nicht mehr transparent, welcher genaue Typ sich hinter der Referenz verbirgt. Damit wird gegen das Liskovsche Substitutionsprinzip verstoßen.

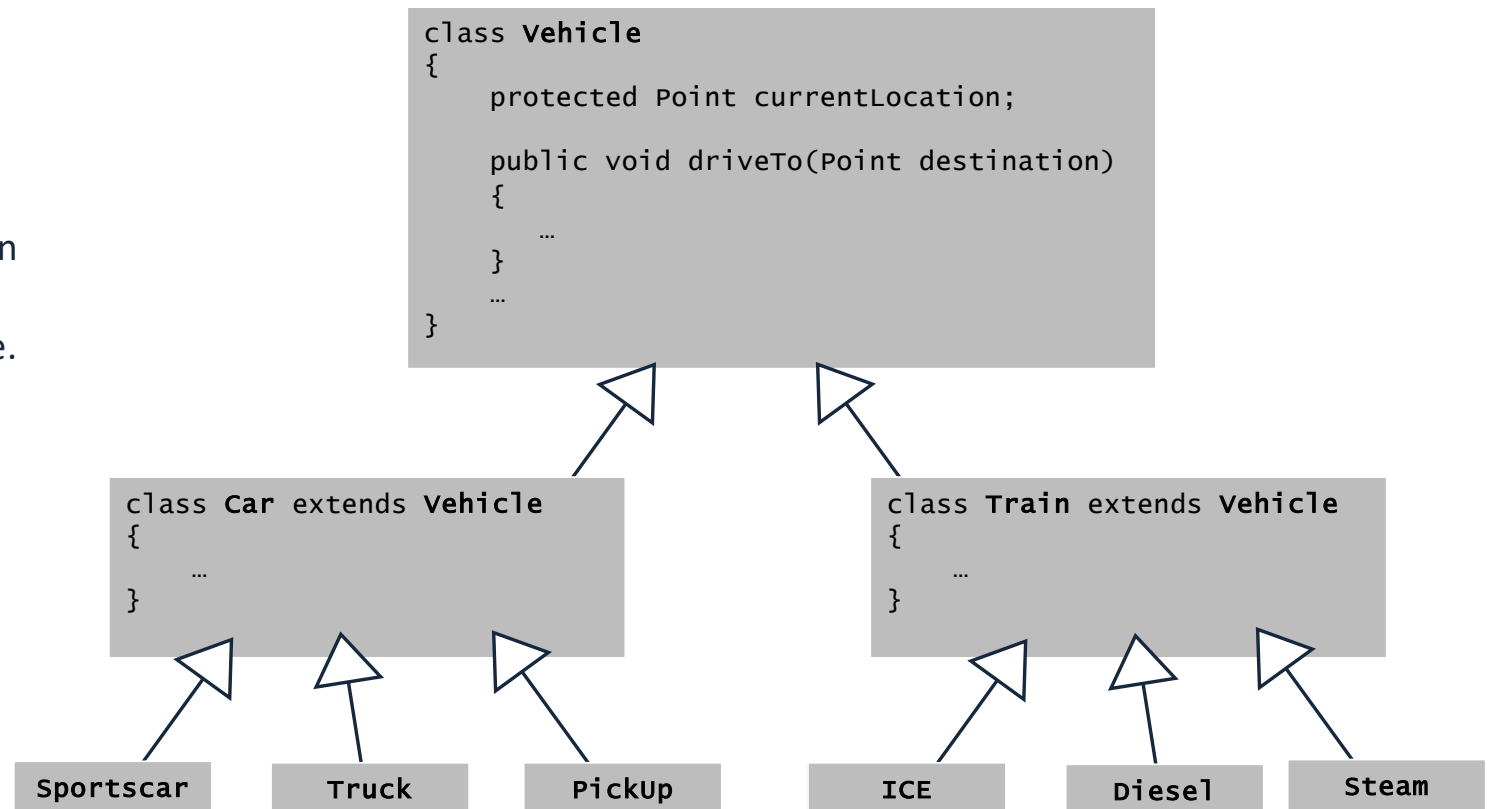
Außerdem ist es kein guter Stil, Objekte auszufragen und dann auf Basis des Ergebnisses unterschiedliche Funktionalität anzubieten. Viel besser wäre es doch, wenn das Objekt wüsste, was zu tun ist.

Polymorphie

Polymorphie

Klassenhierarchie

Um Polymorphie besser verstehen zu können, benötigen wir einige Klassen in einer Klassenhierarchie.



Polymorphie

Klassenhierarchie

```
public static void main(String[] args)
{
    Point home = new Point(0,0);
    Vehicle v = getBestAvailableVehicle();
    v.driveTo(home);
}
```

```
class Vehicle
{
    protected Point currentLocation;

    public void driveTo(Point destination)
    {
        ...
    }
    ...
}
```

Wie wir bereits wissen, kann dann einer Variable der Oberklasse auch eine Instanz jeder Unterklasse zugewiesen werden. Das von `getBestAvailableVehicle()` zurückgelieferte Objekt kann eine beliebige Instanz einer Unterklasse sein. (Zulässiger Upcast / Liskovsches Substitutionsprinzip)

Polymorphie

Überschreiben von Methoden

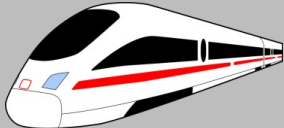
Eine Methode kann in in einer Unterklasse mit einer passenderen Implementierung **überschrieben** werden.

```
class Vehicle
{
    protected Point currentLocation;

    public void driveTo(Point destination)
    {
        turnEngineOn();
        grabSteeringWheel();
        accelerate();
        ...
    }
    ...
}
```



```
class Train extends Vehicle
{
    public void driveTo(Point destination)
    {
        enterCarriage();
        sitDownAndRelax();
        ...
    }
    ...
}
```



Polymorphie

Aufruf überschriebener Methoden

```
public static void main(String[] args)
{
    Point home = new Point(0,0);
    Vehicle v = getBestAvailableVehicle();
    v.driveTo(home);
}
```



Angenommen, die Methode `getBestAvailableVehicle()` liefert einen Zug (Instanz von `Train`) zurück – welche Implementierung von `driveTo()` wird denn dann anschließend aufgerufen?



Polymorphie

Dynamisches Binden

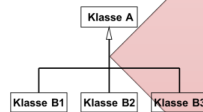
Der Rückgabewert bzw.
die Variable ist **polymorph**
(gr. "vielgestaltig")



Der Compiler akzeptiert den Methodenaufruf, weil die Methode bereits in der Oberklasse Vehicle (= Typ der Referenz) bekannt ist – und damit auch in allen Unterklassen vorhanden sein muss.



Zur Laufzeit wird die Implementierung herangezogen, die für die Instanz am besten passt. Wenn die Methode also überschrieben wurde, dann wird die besser geeignete überschriebene Variante ausgeführt.



Innerhalb der Klassenhierarchie kann die Methode öfters überschrieben werden (z.B. in Train und in ICE). Zum Zuge kommt die „unterste“, am nächsten am Typ der Instanz liegende Variante.

Dynamisches Binden

Polymorphie

Verbesserung

Die Implementierung für das Fahren eines Autos hat in der Oberklasse Vehicle nichts verloren.

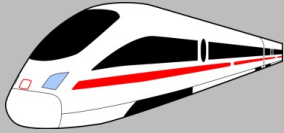
```
class Car extends Vehicle
{
}
}
```

```
class Vehicle
{
    protected Point currentLocation;

    public void driveTo(Point destination)
    {
        turnEngineOn();
        grabSteeringWheel();
        accelerate();
        ...
    }
    ...
}
```



```
class Train extends Vehicle
{
    public void driveTo(Point destination)
    {
        enterCarriage();
        sitDownAndRelax();
        ...
    }
    ...
}
```



Polymorphie

Abstrakte Methoden und Klassen

Abstrakte Methoden dienen als Platzhalter, die in Unterklassen implementiert werden müssen. Sobald eine Klasse eine abstrakte Methode besitzt, ist die Klasse selbst auch eine **abstrakte Klasse**.

```
class Car extends Vehicle
{
    public void driveTo(Point destination)
    {
        turnEngineOn();
        grabSteeringWheel();
        accelerate();
        ...
    }
}
```



```
abstract class Vehicle
{
    protected Point currentLocation;

    abstract public void
        driveTo(Point destination);

    // Es folgt *kein* Methodenrumpf!

    ...
}
```

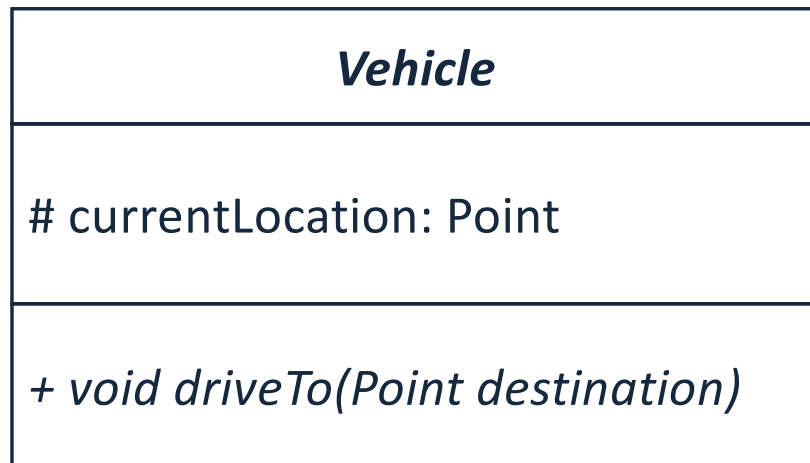
```
class Train extends Vehicle
{
    public void driveTo(Point destination)
    {
        enterCarriage();
        sitDownAndRelax();
        ...
    }
    ...
}
```



Polymorphie

Abstrakte Methoden und Klassen

Abstrakte Klassen und abstrakte Methoden werden in UML kursiv geschrieben.



```
abstract class vehicle
{
    protected Point currentLocation;

    abstract public void
        driveTo(Point destination);

    // Es folgt *kein* Methodenrumpf!

    ...
}
```

Polymorphie

Übersicht

	Vereinbarung	Nutzung	Darf abstrakte Methoden enthalten?	Darf Variablen vereinbaren?	Kann instanziiert werden?
Konkrete Klasse	<pre>class A { ... }</pre>	<pre>class Z extends A { ... }</pre>			
Abstrakte Klasse	<pre>abstract class B { ... }</pre>	<pre>class Z extends B { ... }</pre>			
Interface	<pre>interface C { ... }</pre>	<pre>class Z implements C { ... }</pre>	 *)		

Vielen Dank für die Aufmerksamkeit!

Fragen?